
Relatório de Auditoria de Segurança

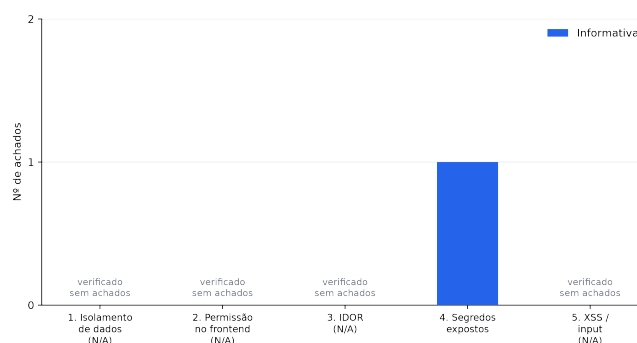
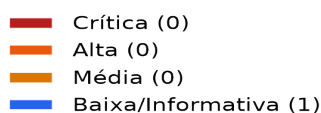
— apostamegasena (Gerador de Apostas)

Auditoria de código-fonte estático · 29/08/2026

Escopo auditado	Repositório completo (branch atual): app Flutter/Dart <i>apostamegasena</i> — código Dart em lib/, configuração de build Android/iOS/Web/Desktop, assets, histórico do git.
Stack detectada	Linguagem: Dart · Framework: Flutter (mobile, com alvos de build adicionais para web/desktop) · Gerência de estado/DI: flutter_bloc + flutter_modular · Persistência: shared_preferences (armazenamento local chave-valor no dispositivo) · Backend/API: nenhum · Autenticação: nenhuma · Banco de dados remoto: nenhum.
Nota metodológica	As 5 categorias do checklist foram mapeadas para esta stack antes da varredura: como o app é 100% local/offline (sem backend, sem contas de usuário, sem rede), as categorias 1 (isolamento de tenant/RLS), 2 (permissão definida no navegador vs. backend) e 3 (IDOR) não têm equivalente aplicável — não existe fronteira de autorização servidor↔cliente nem múltiplos usuários/tenants a isolar. Essas categorias foram verificadas explicitamente como N/A, com a justificativa técnica registrada, em vez de terem achados forçados. As categorias 4 (segredos expostos) e 5 (XSS/input) foram auditadas normalmente sobre código-fonte, configs de build, assets e histórico do git.

Resumo Executivo

A auditoria cobriu as 5 categorias solicitadas, adaptadas à stack real do projeto (ver nota metodológica na capa). O resultado é um app com **superfície de ataque mínima**: não há backend, não há autenticação, não há chamadas de rede em nenhuma linha de código, e não há renderização de HTML/Markdown/WebView. **1 achado informativo** foi registrado (armazenamento local sem criptografia); nenhum achado crítico, alto ou médio foi identificado.



Severidade	Qtd.	Categorias com achado
Crítica	0	—
Alta	0	—
Média	0	—
Baixa / Informativa	1	4. Segredos expostos (nota de robustez, não segredo vazado)
N/A (categoria não se aplica à stack)	3	1. Isolamento de dados · 2. Permissão no frontend · 3. IDOR

Pontos Fortes (verificados)

✓ **Sem segredos no código-fonte** — Varredura por padrões de API key/token/senha/chave privada em lib/, android/, ios/, assets/ e arquivos de configuração de raiz não encontrou nenhuma credencial embutida. O projeto não integra nenhum SDK de terceiros que exigiria chave (ads, analytics, mapas, push).

✓ **Nenhum segredo no histórico do git** — git log --all --diff-filter=A --name-only não retornou nenhum arquivo de nome sensível (.env, key.properties, .jks, .keystore, credential*) já commitado em algum momento do histórico.

✓ **Keystore de release fora do repositório** — android/app/src/./build.gradle.kts lê android/key.properties (listado em android/.gitignore, nunca commitado) para os dados de assinatura de release; sem esse arquivo local o build cai para a chave de debug em vez de falhar de forma insegura ou expor um default público.

✓ **Sem WebView/HTML/Markdown em toda a árvore de widgets** — Nenhuma ocorrência de WebView, flutter_html, markdown, Html.fromHtml, dangerouslySetInnerHTML-equivalente ou `eval` em lib/. Todo

texto é renderizado via widgets Text/TextField nativos do Flutter, que exibem conteúdo como glifos — não há parser de marcação envolvido, logo não há vetor de XSS.

✓ **Campos de entrada restritos a dígitos** — Os dois únicos TextField do app (quantidade de apostas em `generator_page.dart:405` e dezenas manuais em `comparison_page.dart:196`) usam ``FilteringTextInputFormatter.digitsOnly``, então nunca aceitam texto livre/HTML.

✓ **Compartilhamento de texto plano via share_plus** — `generator_page.dart:229` usa ``Share.share(buffer.toString(), ...)``, que entrega texto puro ao share sheet do SO — não há renderização HTML no caminho de compartilhamento.

✓ **Operação por ID confinada ao próprio dispositivo** — ``deleteBetHistory(id)`` em `bet_history_repository.dart:47-54` só opera sobre a lista salva localmente via `SharedPreferences` do próprio aparelho — não existe servidor, conta de usuário ou outro tenant cujo dado poderia ser acessado trocando o id (pré-requisito para IDOR).

✓ **Uso correto de aleatoriedade não-criptográfica** — ``dart:math Random()`` é usado em `generate_bets.dart` para sugerir dezenas de aposta — não há geração de token, senha ou segredo de sessão em nenhum lugar do app, então ``Random()`` (em vez de ``Random.secure()``) é a escolha apropriada.

Pontos Fracos / Riscos Centrais

Nenhum risco de segurança explorável foi identificado. O único ponto registrado é de robustez/defesa em profundidade, não uma vulnerabilidade ativa:

■ **Histórico de apostas em texto plano no dispositivo** — `bet_history_repository.dart` armazena o histórico de apostas geradas via `SharedPreferences` (chave `'bet_history'`) sem criptografia. Isso é aceitável porque o dado não é sensível (apenas os próprios números de aposta gerados pelo usuário — não há PII, login ou dado financeiro), mas fica registrado como nota de robustez para o caso de o escopo do app crescer.

Metodologia — Mapeamento por Categoria

Categoria	Status	Justificativa
1. Banco sem tranca (isolamento de tenant/RLS)	N/A	Mecanismo de isolamento identificado: nenhum. O app não tem backend, banco remoto, Supabase, contas de usuário nem conceito de organização/tenant. Toda a persistência é local, por dispositivo, via SharedPreferences (lib/modules/generator/data/repositories/bet_history_repository.dart) — não existe consulta que precise filtrar por usuário/tenant porque não há mais de um usuário compartilhando a mesma base de dados.
2. Permissão definida no navegador	N/A	Não há papéis (isAdmin, canEdit, role), tela de administração, gestão de usuários nem qualquer operação privilegiada distinta de uso comum. Sem backend, não há endpoint correspondente a validar.
3. IDOR	N/A	Não existem handlers de rota HTTP no projeto (nenhuma dependência http/dio, nenhuma URL literal em lib/). A única operação por ID (deleteBetHistory(id)) foi auditada e confirmada como local ao próprio dispositivo — sem fronteira de autorização a violar.
4. Chaves expostas (hardcode)	Auditado	Varredura por padrões de segredo (api[_]?key, secret, token, password, Bearer, private_key, BEGIN RSA/PRIVATE, Alza, sk_live/sk_test) em lib/, android/, ios/, assets/ e arquivos de configuração de raiz, mais checagem do histórico completo do git por nomes de arquivo sensíveis. Nenhum resultado positivo. Achado informativo registrado sobre armazenamento local não criptografado (ver Pontos Fracos).
5. Inputs sem tratamento (XSS)	N/A	Não há dependência de WebView, flutter_html ou markdown no pubspec.yaml/pubspec.lock; nenhuma ocorrência de renderização de HTML/eval em lib/. O app não tem backend que gere e-mails/templates HTML. Os dois campos de texto livres são restritos a dígitos.

Achados Detalhados

Um único achado foi registrado nesta auditoria, de severidade informativa. As demais categorias não geraram achados — ver justificativas técnicas na seção de Metodologia.

Categoria 4 — Chaves Expostas / Robustez de Dados

Severidade	Arquivo:linha	Descrição
INFORMATIVA	lib/modules/generator/data/repositories/bet_history_repository.dart:11-18	saveBetHistory() grava o histórico de apostas via <code>prefs.setString(_historyKey, jsonString)</code> em texto plano (SharedPreferences não é criptografado no Android/iOS por padrão). Não é uma vulnerabilidade ativa — o dado guardado (números de aposta gerados pelo próprio usuário) não é sensível — mas registra-se como nota de robustez/defesa em profundidade caso o app venha a armazenar dados mais sensíveis no futuro.

Evidência de código

bet_history_repository.dart, linhas 11-18:

```
Future<void> saveBetHistory(BetHistory history) async {
  final prefs = await SharedPreferences.getInstance();
  final currentHistory = await loadBetHistory();
  currentHistory.insert(0, history);
  ...
  final jsonString = jsonEncode(jsonList);
  await prefs.setString(_historyKey, jsonString); // texto plano
}
```

1. Banco sem tranca (isolamento de tenant/RLS) — N/A

Mecanismo de isolamento identificado: nenhum. O app não tem backend, banco remoto, Supabase, contas de usuário nem conceito de organização/tenant. Toda a persistência é local, por dispositivo, via SharedPreferences (lib/modules/generator/data/repositories/bet_history_repository.dart) — não existe consulta que precise filtrar por usuário/tenant porque não há mais de um usuário compartilhando a mesma base de dados.

2. Permissão definida no navegador — N/A

Não há papéis (isAdmin, canEdit, role), tela de administração, gestão de usuários nem qualquer operação privilegiada distinta de uso comum. Sem backend, não há endpoint correspondente a validar.

3. IDOR — N/A

Não existem handlers de rota HTTP no projeto (nenhuma dependência http/dio, nenhuma URL literal em lib/). A única operação por ID (`deleteBetHistory(id)`) foi auditada e confirmada como local ao próprio dispositivo — sem fronteira de autorização a violar.

5. Inputs sem tratamento (XSS) — N/A

Não há dependência de WebView, flutter_html ou markdown no pubspec.yaml/pubspec.lock; nenhuma ocorrência de renderização de HTML/eval em lib/. O app não tem backend que gere e-mails/templates HTML. Os dois campos de texto livres são restritos a dígitos.

Recomendações Priorizadas

P1**Nenhuma ação crítica ou de alta prioridade necessária**

A auditoria não encontrou vulnerabilidades críticas, altas ou médias exploráveis. Não há P1 pendente neste ciclo.

P2**Nenhuma ação de prioridade média pendente**

Idem — sem achados nesta faixa de severidade.

P3**(Opcional) Marcar o histórico local como dado não sensível, ou cifrá-lo**

Caso o app venha a armazenar dados mais sensíveis no futuro (ex.: dados de conta), avaliar migrar o histórico de SharedPreferences em texto plano para um storage cifrado (ex.: pacote flutter_secure_storage) antes disso acontecer. Para o conteúdo atual (números de aposta do próprio usuário), não é urgente.

P3**Manter a disciplina de não commitar segredos**

O projeto já mantém key.properties fora do controle de versão corretamente — manter essa prática ao adicionar qualquer integração futura que exija API key/token.

Issues para o GitHub

Apenas um achado desta auditoria é acionável (os demais são confirmações de N/A ou pontos fortes, que não geram issue). Texto pronto para copiar e colar:

--- ISSUE 1 ---

Título

[Segurança] Histórico de apostas armazenado em texto plano no dispositivo (SharedPreferences)

****Labels sugeridas:**** security, severidade:baixa

****Descrição do problema****

`BetHistoryRepository.saveBetHistory()` (lib/modules/generator/data/repositories/bet_history_repository.dart, linhas 11-18) grava o histórico de apostas via `prefs.setString(\_historyKey, jsonString)`, usando o pacote `shared\_preferences`, que armazena os dados em texto plano no dispositivo (arquivo XML no Android via SharedPreferences padrão, plist no iOS via UserDefaults) sem criptografia.

Isso não é explorável remotamente e o dado atual (números de aposta gerados pelo próprio usuário, sem PII ou credenciais) não é sensível — por isso a severidade é baixa/informativa, não crítica. Registrado como nota de robustez/defesa em profundidade.

****Evidência****

...

lib/modules/generator/data/repositories/bet_history_repository.dart:11-18

```
Future saveBetHistory(BetHistory history) async {
  final prefs = await SharedPreferences.getInstance();
  final currentHistory = await loadBetHistory();
  currentHistory.insert(0, history);
  ...
  final jsonString = jsonEncode(jsonList);
  await prefs.setString(_historyKey, jsonString); // texto plano, sem criptografia
}
```

...

****Impacto****

Baixo no estado atual do app (dado não sensível). Se o app crescer para armazenar dados mais sensíveis (ex.: conta de usuário, dados financeiros reais), esse padrão de armazenamento precisaria ser revisado antes disso — quem tiver acesso físico/root ao dispositivo, ou a um backup não criptografado, pode ler o conteúdo.

****Sugestão de correção****

Nenhuma ação necessária para o escopo atual. Caso o escopo do app mude, considerar migrar para um storage cifrado (ex.: pacote `flutter\_secure\_storage`) para dados que passem a ser sensíveis, mantendo `shared\_preferences` apenas para preferências não sensíveis (tema, configurações de UI, etc.).

****Critérios de aceite****

- [] Decisão registrada: manter `shared\_preferences` para o histórico de apostas (dado não sensível) OU migrar para storage cifrado, caso o escopo do app inclua dados sensíveis no futuro
- [] Se migrar: `flutter\_secure\_storage` (ou equivalente) adotado para qualquer dado que envolva PII/credenciais/dados financeiros reais
- [] Nenhuma regressão nos testes existentes de `BetHistoryRepository`

--- FIM ISSUE 1 ---